

Prototype — Bilingual Study Document (English + Hindi)

Source: refactoring.guru/design-patterns/prototype — content and order preserved

Study Companion · Updated July 2026

Prototype

Also known as: Clone

अन्य नाम: क्लोन (Clone)

Source note / स्रोत टिप्पणी: This document is built directly from refactoring.guru's Prototype pattern page (English text preserved paragraph-by-paragraph, with a Hindi translation/explanation immediately below each one, in the same order as the source). PHP code is included in this same file, as requested.

यह दस्तावेज़ (document) सीधे refactoring.guru के Prototype पैटर्न पेज से बनाया गया है (अंग्रेज़ी टेक्स्ट को पैराग्राफ़-दर-पैराग्राफ़ रखा गया है, और हर पैराग्राफ़ के ठीक नीचे उसी क्रम में हिंदी अनुवाद/व्याख्या दी गई है)। PHP कोड इसी फ़ाइल में शामिल किया गया है, जैसा अनुरोध किया गया था।

Intent

Prototype is a creational design pattern that lets you copy existing objects without making your code dependent on their classes.

प्रोटोटाइप (Prototype) एक **क्रिएशनल डिज़ाइन पैटर्न (creational design pattern)** है, जो आपको मौजूदा ऑब्जेक्ट्स (existing objects) की कॉपी बनाने की सुविधा देता है — बिना अपने कोड को उनकी क्लासेज़ (classes) पर निर्भर बनाए।

Problem

Say you have an object, and you want to create an exact copy of it. How would you do it? First, you have to create a new object of the same class. Then you have to go through all the fields of the original object and copy their values over to the new object.

मान लीजिए आपके पास एक ऑब्जेक्ट है, और आप उसकी बिल्कुल सटीक (exact) कॉपी बनाना चाहते हैं। आप यह कैसे करेंगे? सबसे पहले, आपको उसी क्लास का एक नया ऑब्जेक्ट बनाना होगा। फिर आपको मूल (original) ऑब्जेक्ट की सभी फ़िल्ड्स (fields) में जाकर उनकी वैल्यूज़ (values) को नए ऑब्जेक्ट में कॉपी करना होगा।

Nice! But there's a catch. Not all objects can be copied that way because some of the object's fields may be private and not visible from outside of the object itself.

बढ़िया! लेकिन इसमें एक पेच (catch) है। सभी ऑब्जेक्ट्स को इस तरह कॉपी नहीं किया जा सकता, क्योंकि ऑब्जेक्ट के कुछ फ़िल्ड्स **प्राइवेट (private)** हो सकते हैं और वे ऑब्जेक्ट के बाहर से दिखाई ही नहीं देते।

Copying an object "from the outside" isn't always possible.

किसी ऑब्जेक्ट को "बाहर से" (from the outside) कॉपी करना हमेशा संभव नहीं होता।

There's one more problem with the direct approach. Since you have to know the object's class to create a duplicate, your code becomes dependent on that class. If the extra dependency doesn't scare you, there's another catch. Sometimes you only know the interface that the object follows, but not its concrete class, when, for example, a parameter in a method accepts any objects that follow some interface.

डायरेक्ट (direct) तरीके में एक और समस्या है। चूँकि डुप्लीकेट (duplicate) बनाने के लिए आपको ऑब्जेक्ट की क्लास पता होनी चाहिए, इसलिए आपका कोड उस क्लास पर **निर्भर (dependent)** हो जाता है। अगर यह अतिरिक्त **डिपेंडेंसी (dependency)** आपको परेशान नहीं करती, तो एक और पेच है। कभी-कभी आपको सिर्फ़ वह **इंटरफ़ेस (interface)** पता होता है जिसे ऑब्जेक्ट फ़ॉलो करता है, लेकिन उसकी असली (concrete) क्लास पता नहीं होती — जैसे, जब किसी मेथड (method) का पैरामीटर (parameter) किसी भी ऐसे ऑब्जेक्ट को स्वीकार करता है जो किसी इंटरफ़ेस को फ़ॉलो करता हो।

Solution

The Prototype pattern delegates the cloning process to the actual objects that are being cloned. The pattern declares a common interface for all objects that support cloning. This interface lets you clone an object without coupling your code to the class of that object. Usually, such an interface contains just a single `clone` method.

प्रोटोटाइप पैटर्न, **क्लोनिंग (cloning)** की पूरी प्रक्रिया को उन्हीं असली ऑब्जेक्ट्स को सौंप (delegate) देता है जिन्हें क्लोन किया जा रहा है। यह पैटर्न सभी क्लोनिंग-सपोर्टेड ऑब्जेक्ट्स के लिए एक **कॉमन इंटरफ़ेस (common interface)** डिक्लेयर करता है। यह इंटरफ़ेस आपको किसी ऑब्जेक्ट को क्लोन करने देता है, बिना अपने कोड को उस ऑब्जेक्ट की क्लास से **कपल (couple)** किए। आमतौर पर, ऐसे इंटरफ़ेस में सिर्फ़ एक ही `clone` मेथड होता है।

The implementation of the `clone` method is very similar in all classes. The method creates an object of the current class and carries over all of the field values of the old object into the new one. You can even copy private fields because most programming languages let objects access private fields of other objects that belong to the same class.

`clone` मेथड का **इम्प्लीमेंटेशन (implementation)** लगभग सभी क्लासेज़ में एक जैसा होता है। यह मेथड करंट क्लास (current class) का एक ऑब्जेक्ट बनाता है और पुराने ऑब्जेक्ट की सभी फ़िल्ड वैल्यूज़ को नए ऑब्जेक्ट में कॉपी कर देता है। आप प्राइवेट फ़िल्ड्स भी कॉपी कर सकते हैं, क्योंकि ज़्यादातर प्रोग्रामिंग लैंग्वेजेज़ (programming languages) ऑब्जेक्ट्स को उसी क्लास के दूसरे ऑब्जेक्ट्स के प्राइवेट फ़िल्ड्स तक पहुँचने (access) की अनुमति देती हैं।

An object that supports cloning is called a *prototype*. When your objects have dozens of fields and hundreds of possible configurations, cloning them might serve as an alternative to subclassing.

जो ऑब्जेक्ट क्लोनिंग को सपोर्ट करता है उसे **प्रोटोटाइप (prototype)** कहा जाता है। जब आपके ऑब्जेक्ट्स में दर्जनों फ़िल्ड्स और सैकड़ों संभावित **कॉन्फ़िगरेशन्स (configurations)** हों, तो उन्हें क्लोन करना **सबक्लासिंग (subclassing)** का एक विकल्प (alternative) बन सकता है।

Pre-built prototypes can be an alternative to subclassing.

पहले से बने (pre-built) प्रोटोटाइप्स, सबक्लासिंग का एक विकल्प हो सकते हैं।

Here's how it works: you create a set of objects, configured in various ways. When you need an object like the one you've configured, you just clone a prototype instead of constructing a new object from scratch.

यह इस तरह काम करता है: आप ऑब्जेक्ट्स का एक सेट (set) बनाते हैं, जो अलग-अलग तरीकों से कॉन्फ़िगर (configure) किए गए होते हैं। जब आपको वैसा ही कोई ऑब्जेक्ट चाहिए जैसा आपने पहले से कॉन्फ़िगर किया है, तो आप बस एक प्रोटोटाइप को क्लोन कर लेते हैं, बजाय इसके कि शुरू से (from scratch) एक नया ऑब्जेक्ट बनाएँ।

Real-World Analogy

In real life, prototypes are used for performing various tests before starting mass production of a product. However, in this case, prototypes don't participate in any actual production, playing a passive role instead.

असल ज़िंदगी में, किसी प्रोडक्ट (product) के **मास प्रोडक्शन (mass production)** से पहले, विभिन्न टेस्ट्स करने के लिए प्रोटोटाइप्स का उपयोग किया जाता है। हालाँकि, इस स्थिति में प्रोटोटाइप्स असली प्रोडक्शन में भाग नहीं लेते, बल्कि एक **पैसिव (passive)** भूमिका निभाते हैं।

The division of a cell.

एक कोशिका (cell) का विभाजन।

Since industrial prototypes don't really copy themselves, a much closer analogy to the pattern is the process of mitotic cell division (biology, remember?). After mitotic division, a pair of identical cells is formed. The original cell acts as a prototype and takes an active role in creating the copy.

चूँकि औद्योगिक (industrial) प्रोटोटाइप्स असल में खुद की कॉपी नहीं बनाते, इसलिए इस पैटर्न की कहीं ज़्यादा नज़दीकी उपमा (analogy) **माइटोटिक कोशिका विभाजन (mitotic cell division)** की प्रक्रिया है (जीव विज्ञान, याद है?)। माइटोटिक डिवीज़न के बाद, दो बिल्कुल एक जैसी (identical) कोशिकाओं की जोड़ी बनती है। मूल कोशिका एक प्रोटोटाइप की तरह काम करती है और कॉपी बनाने में **सक्रिय (active)** भूमिका निभाती है।

Structure

Basic implementation

बेसिक इम्प्लीमेंटेशन (Basic Implementation)

1. The **Prototype** interface declares the cloning methods. In most cases, it's a single `clone` method.

Prototype इंटरफ़ेस क्लोनिंग मेथड्स को डिक्लेयर करता है। ज़्यादातर मामलों में, यह एक सिंगल `clone` मेथड होता है।

1. The **Concrete Prototype** class implements the cloning method. In addition to copying the original object's data to the clone, this method may also handle some edge cases of the cloning process related to cloning linked objects, untangling recursive dependencies, etc.

Concrete Prototype क्लास क्लोनिंग मेथड को इम्प्लीमेंट करती है। मूल ऑब्जेक्ट का डेटा क्लोन में कॉपी करने के अलावा, यह मेथड क्लोनिंग प्रोसेस के कुछ **एज केसेज़ (edge cases)** को भी संभाल सकता है — जैसे लिंकड ऑब्जेक्ट्स (linked objects) को क्लोन करना, **रिकर्सिव डिपेंडेंसीज़ (recursive dependencies)** को सुलझाना, आदि।

1. The **Client** can produce a copy of any object that follows the prototype interface.

Client, prototype इंटरफ़ेस को फ़ॉलो करने वाले किसी भी ऑब्जेक्ट की कॉपी बना सकता है।

Prototype registry implementation

प्रोटोटाइप रजिस्ट्री इम्प्लीमेंटेशन (Prototype Registry Implementation)

1. The **Prototype Registry** provides an easy way to access frequently-used prototypes. It stores a set of pre-built objects that are ready to be copied. The simplest prototype registry is a `name → prototype` hash map. However, if you need better search criteria than a simple name, you can build a much more robust version of the registry.

Prototype Registry, बार-बार इस्तेमाल होने वाले प्रोटोटाइप्स तक पहुँचने का एक आसान तरीका देती है। यह पहले से बने (pre-built) ऑब्जेक्ट्स का एक सेट स्टोर करती है, जो कॉपी करने के लिए तैयार होते हैं। सबसे सरल prototype registry एक `name → prototype` **हैश मैप (hash map)** होती है। हालाँकि, अगर आपको सिर्फ़ एक साधारण नाम (name) से बेहतर **सर्च क्राइटेरिया (search criteria)** चाहिए, तो आप रजिस्ट्री का एक कहीं ज़्यादा मज़बूत (robust) वर्शन बना सकते हैं।

Pseudocode

In this example, the **Prototype** pattern lets you produce exact copies of geometric objects, without coupling the code to their classes.

इस उदाहरण में, प्रोटोटाइप पैटर्न आपको ज्यामितीय (geometric) ऑब्जेक्ट्स की सटीक कॉपी बनाने देता है, बिना कोड को उनकी क्लासेज़ से जोड़े (coupling) रखे।

Cloning a set of objects that belong to a class hierarchy.

एक क्लास हायरार्की (class hierarchy) से जुड़े ऑब्जेक्ट्स के एक सेट को क्लोन करना।

All shape classes follow the same interface, which provides a cloning method. A subclass may call the parent's cloning method before copying its own field values to the resulting object.

सभी शेप (shape) क्लासेज़ एक ही इंटरफ़ेस को फ़ॉलो करती हैं, जो एक क्लोनिंग मेथड मुहैया कराता है। एक सबक्लास, अपनी खुद की फ़ील्ड वैल्यूज़ को रिज़ल्टिंग (resulting) ऑब्जेक्ट में कॉपी करने से पहले, पैरेंट (parent) के क्लोनिंग मेथड को कॉल कर सकती है।

(Pseudocode preserved exactly as in the source — यह पूडोकोड (pseudocode) स्रोत से बिल्कुल वैसा ही रखा गया है)

```

// Base prototype.
abstract class Shape is
    field X: int
    field Y: int
    field color: string

    // A regular constructor.
    constructor Shape() is
        // ...

    // The prototype constructor. A fresh object is initialized
    // with values from the existing object.
    constructor Shape(source: Shape) is
        this()
        this.X = source.X
        this.Y = source.Y
        this.color = source.color

    // The clone operation returns one of the Shape subclasses.
    abstract method clone():Shape

// Concrete prototype. The cloning method creates a new object
// in one go by calling the constructor of the current class and
// passing the current object as the constructor's argument.
// Performing all the actual copying in the constructor helps to
// keep the result consistent: the constructor will not return a
// result until the new object is fully built; thus, no object
// can have a reference to a partially-built clone.
class Rectangle extends Shape is
    field width: int
    field height: int

    constructor Rectangle(source: Rectangle) is
        // A parent constructor call is needed to copy private
        // fields defined in the parent class.
        super(source)
        this.width = source.width
        this.height = source.height

    method clone():Shape is
        return new Rectangle(this)

class Circle extends Shape is
    field radius: int

    constructor Circle(source: Circle) is
        super(source)
        this.radius = source.radius

    method clone():Shape is
        return new Circle(this)

// Somewhere in the client code.
class Application is
    field shapes: array of Shape

    constructor Application() is
        Circle circle = new Circle()

```



```

circle.X = 10
circle.Y = 10
circle.radius = 20
shapes.add(circle)

Circle anotherCircle = circle.clone()
shapes.add(anotherCircle)
// The `anotherCircle` variable contains an exact copy
// of the `circle` object.

Rectangle rectangle = new Rectangle()
rectangle.width = 10
rectangle.height = 20
shapes.add(rectangle)

method businessLogic() is
// Prototype rocks because it lets you produce a copy of
// an object without knowing anything about its type.
Array shapesCopy = new Array of Shapes.

// For instance, we don't know the exact elements in the
// shapes array. All we know is that they are all
// shapes. But thanks to polymorphism, when we call the
// `clone` method on a shape the program checks its real
// class and runs the appropriate clone method defined
// in that class. That's why we get proper clones
// instead of a set of simple Shape objects.
foreach (s in shapes) do
    shapesCopy.add(s.clone())

// The `shapesCopy` array contains exact copies of the
// `shape` array's children.

```

Same logic, in PHP (added for this study document) / यही लॉजिक, PHP में (इस दस्तावेज़ के लिए जोड़ा गया):

```

<?php

// Base prototype. / बेस प्रोटोटाइप।
abstract class Shape
{
    public int $x;
    public int $y;
    public string $color;

    // A regular constructor. / एक सामान्य कंस्ट्रक्टर।
    public function __construct()
    {
    }

    // The prototype constructor: initializes a fresh object with
    // values copied from the existing ($source) object.
    // प्रोटोटाइप कंस्ट्रक्टर: मौजूदा ($source) ऑब्जेक्ट से वैल्यूज़ लेकर
    // एक नए ऑब्जेक्ट को इनिशियलाइज़ करता है।
    protected function initFrom(Shape $source): void
    {
        $this->x = $source->x;
        $this->y = $source->y;
        $this->color = $source->color;
    }

    // Every subclass must implement its own clone.
    // हर सबक्लास को अपना खुद का clone इम्प्लीमेंट करना ज़रूरी है।
    abstract public function clone(): Shape;
}

// Concrete prototype: Rectangle
// कॉन्क्रीट प्रोटोटाइप: Rectangle
class Rectangle extends Shape
{
    public int $width;
    public int $height;

    public static function fromSource(Rectangle $source): self
    {
        $instance = new self();
        $instance->initFrom($source);           // copy parent's private-ish fields
        $instance->width = $source->width;
        $instance->height = $source->height;
        return $instance;
    }

    public function clone(): Shape
    {
        return self::fromSource($this);
    }
}

// Concrete prototype: Circle
// कॉन्क्रीट प्रोटोटाइप: Circle
class Circle extends Shape
{
    public int $radius;

```

```

public static function fromSource(Circle $source): self
{
    $instance = new self();
    $instance->initFrom($source);
    $instance->radius = $source->radius;
    return $instance;
}

public function clone(): Shape
{
    return self::fromSource($this);
}
}

// Client code / क्लाइंट कोड
final class Application
{
    /** @var Shape[] */
    private array $shapes = [];

    public function __construct()
    {
        $circle = new Circle();
        $circle->x = 10;
        $circle->y = 10;
        $circle->radius = 20;
        $circle->color = "red";
        $this->shapes[] = $circle;

        $anotherCircle = $circle->clone();
        $this->shapes[] = $anotherCircle;
        // $anotherCircle contains an exact copy of $circle.
        // $anotherCircle में $circle की बिल्कुल सटीक कॉपी है।

        $rectangle = new Rectangle();
        $rectangle->width = 10;
        $rectangle->height = 20;
        $rectangle->color = "blue";
        $this->shapes[] = $rectangle;
    }

    public function businessLogic(): void
    {
        $shapesCopy = [];

        // We don't know the exact concrete type of each $s – only that
        // it's a Shape. Polymorphism ensures the correct clone() runs.
        // हमें हर $s की असली (concrete) टाइप नहीं पता – बस इतना पता है
        // कि यह एक Shape है। पॉलिमॉर्फिज़्म (polymorphism) की वजह से
        // सही clone() मेटड ही चलता है।
        foreach ($this->shapes as $s) {
            $shapesCopy[] = $s->clone();
        }

        foreach ($shapesCopy as $i => $copy) {
            printf(
                "Copy #%d: %s (x=%d, y=%d, color=%s)\n",
                $i,
                get_class($copy),
            );
        }
    }
}

```

```
        $copy->x,  
        $copy->y,  
        $copy->color  
    );  
    }  
}  
  
(new Application())->businessLogic();
```

Applicability

Use the Prototype pattern when your code shouldn't depend on the concrete classes of objects that you need to copy.

प्रोटोटाइप पैटर्न का उपयोग तब करें जब आपके कोड को उन ऑब्जेक्ट्स की असली (concrete) क्लासेज़ पर निर्भर नहीं होना चाहिए, जिन्हें आपको कॉपी करना है।

This happens a lot when your code works with objects passed to you from 3rd-party code via some interface. The concrete classes of these objects are unknown, and you couldn't depend on them even if you wanted to.

ऐसा तब अक्सर होता है जब आपका कोड उन ऑब्जेक्ट्स के साथ काम करता है जो थर्ड-पार्टी (3rd-party) कोड से किसी इंटरफ़ेस के ज़रिए आपको पास किए जाते हैं। इन ऑब्जेक्ट्स की असली क्लासेज़ अज्ञात (unknown) होती हैं, और आप चाहकर भी उन पर निर्भर नहीं हो सकते।

The Prototype pattern provides the client code with a general interface for working with all objects that support cloning. This interface makes the client code independent from the concrete classes of objects that it clones.

प्रोटोटाइप पैटर्न, क्लाइंट कोड (client code) को उन सभी ऑब्जेक्ट्स के साथ काम करने के लिए एक जनरल (general) इंटरफ़ेस देता है जो क्लोनिंग को सपोर्ट करते हैं। यह इंटरफ़ेस क्लाइंट कोड को उन ऑब्जेक्ट्स की असली क्लासेज़ से **स्वतंत्र (independent)** बना देता है जिन्हें वह क्लोन करता है।

Use the pattern when you want to reduce the number of subclasses that only differ in the way they initialize their respective objects.

इस पैटर्न का उपयोग तब करें जब आप उन सबक्लासेज़ (subclasses) की संख्या घटाना चाहते हैं जो केवल इस बात में अलग होती हैं कि वे अपने-अपने ऑब्जेक्ट्स को कैसे इनिशियलाइज़ (initialize) करती हैं।

Suppose you have a complex class that requires a laborious configuration before it can be used. There are several common ways to configure this class, and this code is scattered through your app. To reduce the duplication, you create several subclasses and put every common configuration code into their constructors. You solved the duplication problem, but now you have lots of dummy subclasses.

मान लीजिए आपके पास एक कॉम्प्लेक्स (complex) क्लास है, जिसका उपयोग करने से पहले एक मेहनती (laborious) कॉन्फ़िगरेशन ज़रूरी है। इस क्लास को कॉन्फ़िगर करने के कई सामान्य तरीके हैं, और यह कोड आपकी पूरी ऐप (app) में बिखरा (scattered) हुआ है। **डुप्लिकेशन (duplication)** घटाने के लिए, आप कई सबक्लासेज़ बनाते हैं और हर सामान्य कॉन्फ़िगरेशन कोड को उनके कंस्ट्रक्टर (constructors) में डाल देते हैं। आपने डुप्लिकेशन की समस्या तो हल कर दी, लेकिन अब आपके पास ढेर सारी **डमी (dummy)** सबक्लासेज़ हैं।

The Prototype pattern lets you use a set of pre-built objects configured in various ways as prototypes. Instead of instantiating a subclass that matches some configuration, the client can simply look for an appropriate prototype and clone it.

प्रोटोटाइप पैटर्न आपको अलग-अलग तरीकों से कॉन्फ़िगर किए गए पहले से बने ऑब्जेक्ट्स के एक सेट को प्रोटोटाइप्स की तरह इस्तेमाल करने देता है। किसी कॉन्फ़िगरेशन से मेल खाने वाली सबक्लास को **इंस्टैंशिएट (instantiate)** करने के बजाय, क्लाइंट बस एक उपयुक्त (appropriate) प्रोटोटाइप ढूँढ़ सकता है और उसे क्लोन कर सकता है।

How to Implement

1. Create the prototype interface and declare the `clone` method in it. Or just add the method to all classes of an existing class hierarchy, if you have one.

prototype इंटरफ़ेस बनाएँ और उसमें `clone` मेथड डिक्लेयर करें। या अगर आपके पास पहले से एक क्लास हायरार्की है, तो बस उसकी सभी क्लासेज़ में यह मेथड जोड़ दें।

1. A prototype class must define the alternative constructor that accepts an object of that class as an argument. The constructor must copy the values of all fields defined in the class from the passed object into the newly created instance. If you're changing a subclass, you must call the parent constructor to let the superclass handle the cloning of its private fields.

एक prototype क्लास को एक वैकल्पिक (alternative) कंस्ट्रक्टर डिफ़ाइन करना चाहिए, जो उसी क्लास के एक ऑब्जेक्ट को आर्गुमेंट (argument) के रूप में स्वीकार करे। यह कंस्ट्रक्टर, पास किए गए ऑब्जेक्ट से क्लास में डिफ़ाइन सभी फ़िल्ड्स की वैल्यूज़ को नए बने इंस्टेंस (instance) में कॉपी करे। अगर आप किसी सबक्लास में बदलाव कर रहे हैं, तो आपको पैरेंट कंस्ट्रक्टर को कॉल करना ज़रूरी है, ताकि सुपरक्लास (superclass) अपने प्राइवेट फ़िल्ड्स की क्लोनिंग खुद संभाल सके।

If your programming language doesn't support method overloading, you won't be able to create a separate "prototype" constructor. Thus, copying the object's data into the newly created clone will have to be performed within the `clone` method. Still, having this code in a regular constructor is safer because the resulting object is returned fully configured right after you call the `new` operator.

अगर आपकी प्रोग्रामिंग लैंग्वेज **मेथड ओवरलोडिंग (method overloading)** सपोर्ट नहीं करती, तो आप एक अलग "prototype" कंस्ट्रक्टर नहीं बना पाएँगे। इसलिए, ऑब्जेक्ट का डेटा नए बने क्लोन में कॉपी करने का काम `clone` मेथड के अंदर ही करना होगा। फिर भी, इस कोड को एक सामान्य कंस्ट्रक्टर में रखना ज़्यादा सुरक्षित (safer) है, क्योंकि `new` ऑपरेटर कॉल करते ही रिज़ल्टिंग ऑब्जेक्ट पूरी तरह कॉन्फ़िगर होकर वापस मिलता है।

1. The cloning method usually consists of just one line: running a `new` operator with the prototypical version of the constructor. Note, that every class must explicitly override the cloning method and use its own class name along with the `new` operator. Otherwise, the cloning method may produce an object of a parent class.

क्लोनिंग मेथड में आमतौर पर सिर्फ़ एक ही लाइन होती है: कंस्ट्रक्टर के prototypical वर्शन के साथ `new` ऑपरेटर चलाना। ध्यान दें कि हर क्लास को क्लोनिंग मेथड को स्पष्ट रूप से **ओवरराइड (override)** करना चाहिए और `new` ऑपरेटर के साथ अपनी खुद की क्लास का नाम इस्तेमाल करना चाहिए। ऐसा न करने पर, क्लोनिंग मेथड पैरेंट क्लास का ऑब्जेक्ट बना सकता है।

1. Optionally, create a centralized prototype registry to store a catalog of frequently used prototypes.

वैकल्पिक रूप से (optionally), बार-बार इस्तेमाल होने वाले प्रोटोटाइप्स की सूची (catalog) स्टोर करने के लिए एक **केंद्रीकृत (centralized)** prototype registry बनाएँ।

You can implement the registry as a new factory class or put it in the base prototype class with a static method for fetching the prototype. This method should search for a prototype based on

search criteria that the client code passes to the method. The criteria might either be a simple string tag or a complex set of search parameters. After the appropriate prototype is found, the registry should clone it and return the copy to the client.

आप इस रजिस्ट्री को एक नई फैक्टरी (factory) क्लास के रूप में इम्प्लीमेंट कर सकते हैं, या इसे बेस prototype क्लास में एक स्टैटिक (static) मेथड के साथ रख सकते हैं, जो प्रोटोटाइप को फ़ेच (fetch) करे। इस मेथड को क्लाइंट कोड द्वारा भेजे गए सर्च क्राइटेरिया के आधार पर प्रोटोटाइप ढूँढ़ना चाहिए। यह क्राइटेरिया एक साधारण स्ट्रिंग टैग (string tag) या सर्च पैरामीटर्स (parameters) का एक जटिल (complex) सेट हो सकता है। उपयुक्त प्रोटोटाइप मिल जाने के बाद, रजिस्ट्री को उसे क्लोन करके कॉपी क्लाइंट को वापस देनी चाहिए।

Finally, replace the direct calls to the subclasses' constructors with calls to the factory method of the prototype registry.

अंत में, सबक्लासेज़ के कंस्ट्रक्टर्स को सीधे कॉल करने की जगह, prototype registry के फैक्टरी मेथड को कॉल करें।

Pros and Cons

Pros / फ़ायदे

- You can clone objects without coupling to their concrete classes. आप ऑब्जेक्ट्स को उनकी असली क्लासेज़ से जुड़े (coupled) बिना क्लोन कर सकते हैं।
- You can get rid of repeated initialization code in favor of cloning pre-built prototypes. आप बार-बार दोहराए जाने वाले इनिशियलाइज़ेशन (initialization) कोड से छुटकारा पा सकते हैं, क्योंकि पहले से बने प्रोटोटाइप्स को क्लोन किया जा सकता है।
- You can produce complex objects more conveniently. आप कॉम्प्लेक्स ऑब्जेक्ट्स को कहीं ज़्यादा आसानी (conveniently) से बना सकते हैं।
- You get an alternative to inheritance when dealing with configuration presets for complex objects. कॉम्प्लेक्स ऑब्जेक्ट्स के लिए कॉन्फ़िगरेशन प्रीसेट्स (presets) से निपटते समय, आपको इनहेरिटेंस (inheritance) का एक विकल्प मिल जाता है।

Cons / नुक़सान

- Cloning complex objects that have circular references might be very tricky. जिन कॉम्प्लेक्स ऑब्जेक्ट्स में **सर्कुलर रेफ़रेंस (circular references)** होते हैं, उन्हें क्लोन करना काफ़ी मुश्किल (tricky) हो सकता है।
-

Relations with Other Patterns

1. Many designs start by using Factory Method (less complicated and more customizable via subclasses) and evolve toward Abstract Factory, Prototype, or Builder (more flexible, but more complicated).

कई डिज़ाइन्स की शुरुआत Factory Method (जो कम जटिल है और सबक्लासेज़ के ज़रिए ज़्यादा कस्टमाइज़ेबल (customizable) है) से होती है, और धीरे-धीरे Abstract Factory, Prototype, या Builder (जो ज़्यादा फ़्लेक्सिबल (flexible) हैं, लेकिन ज़्यादा जटिल भी हैं) की तरफ़ बढ़ती हैं।

1. Abstract Factory classes are often based on a set of Factory Methods, but you can also use Prototype to compose the methods on these classes.

Abstract Factory क्लासेज़ अक्सर कई Factory Methods के सेट पर आधारित होती हैं, लेकिन आप इन क्लासेज़ के मेथड्स को **कंपोज़ (compose)** करने के लिए Prototype का भी उपयोग कर सकते हैं।

1. Prototype can help when you need to save copies of Commands into history.

जब आपको Commands की कॉपीज़ को हिस्ट्री (history) में सेव करना हो, तब Prototype मददगार हो सकता है।

1. Designs that make heavy use of Composite and Decorator can often benefit from using Prototype. Applying the pattern lets you clone complex structures instead of re-constructing them from scratch.

जो डिज़ाइन्स Composite और Decorator का भारी उपयोग करते हैं, उन्हें अक्सर Prototype इस्तेमाल करने से फ़ायदा हो सकता है। इस पैटर्न को लागू (apply) करने से आप कॉम्प्लेक्स स्ट्रक्चर्स (structures) को शुरू से दोबारा बनाने के बजाय क्लोन कर सकते हैं।

1. Prototype isn't based on inheritance, so it doesn't have its drawbacks. On the other hand, Prototype requires a complicated initialization of the cloned object. Factory Method is based on inheritance but doesn't require an initialization step.

Prototype इनहेरिटेंस पर आधारित नहीं है, इसलिए इसमें इनहेरिटेंस की कमियाँ (drawbacks) नहीं होतीं। दूसरी ओर, Prototype में क्लोन किए गए ऑब्जेक्ट का इनिशियलाइज़ेशन जटिल होता है। Factory Method इनहेरिटेंस पर आधारित है, लेकिन इसमें इनिशियलाइज़ेशन स्टेप की ज़रूरत नहीं पड़ती।

1. Sometimes Prototype can be a simpler alternative to Memento. This works if the object, the state of which you want to store in the history, is fairly straightforward and doesn't have links to external resources, or the links are easy to re-establish.

कभी-कभी Prototype, Memento का एक सरल विकल्प हो सकता है। यह तब काम करता है जब वह ऑब्जेक्ट — जिसकी स्टेट (state) आप हिस्ट्री में स्टोर करना चाहते हैं — काफ़ी सीधा-सादा (straightforward) हो और उसके बाहरी रिसोर्सज़ (external resources) से लिंक न हों, या वे लिंक्स दोबारा आसानी से बनाए जा सकते हों।

1. Abstract Factories, Builders and Prototypes can all be implemented as Singletons.

Abstract Factories, Builders और Prototypes — इन सभी को Singletons के रूप में इम्प्लीमेंट किया जा सकता है।

Code Examples

Usage examples: The Prototype pattern is available in PHP out of the box. You can use the `clone` keyword to create an exact copy of an object. To add cloning support to a class, you need to implement a `__clone` method.

उपयोग के उदाहरण: प्रोटोटाइप पैटर्न PHP में पहले से ही उपलब्ध (out of the box) है। आप किसी ऑब्जेक्ट की सटीक कॉपी बनाने के लिए `clone` कीवर्ड (keyword) का उपयोग कर सकते हैं। किसी क्लास में क्लोनिंग सपोर्ट जोड़ने के लिए, आपको एक `__clone` मेथड इम्प्लीमेंट करना होगा।

Identification: The prototype can be easily recognized by a `clone` or `copy` methods, etc.

पहचान (Identification): prototype को `clone` या `copy` जैसे मेथड्स के ज़रिए आसानी से पहचाना जा सकता है।

Conceptual Example

This example illustrates the structure of the Prototype design pattern and focuses on the following questions: What classes does it consist of? What roles do these classes play? In what way the elements of the pattern are related? After learning about the pattern's structure it'll be easier for you to grasp the following example, based on a real-world PHP use case.

यह उदाहरण प्रोटोटाइप डिज़ाइन पैटर्न की संरचना (structure) को दिखाता है और इन सवालों पर ध्यान केंद्रित करता है: इसमें कौन-सी क्लासेज़ शामिल हैं? ये क्लासेज़ कौन-सी भूमिकाएँ (roles) निभाती हैं? पैटर्न के एलिमेंट्स (elements) आपस में किस तरह जुड़े (related) हैं? पैटर्न की संरचना समझने के बाद, आपको आगे दिया गया उदाहरण समझना आसान होगा, जो एक असली दुनिया (real-world) के PHP उपयोग-मामले (use case) पर आधारित है।

index.php — Conceptual example (verbatim from source / स्रोत से ज्यों-का-त्यों):

```

<?php

namespace RefactoringGuru\Prototype\Conceptual;

/**
 * The example class that has cloning ability. We'll see how the values of field
 * with different types will be cloned.
 */
class Prototype
{
    public $primitive;
    public $component;
    public $circularReference;

    /**
     * PHP has built-in cloning support. You can `clone` an object without
     * defining any special methods as long as it has fields of primitive types.
     * Fields containing objects retain their references in a cloned object.
     * Therefore, in some cases, you might want to clone those referenced
     * objects as well. You can do this in a special `__clone()` method.
     */
    public function __clone()
    {
        $this->component = clone $this->component;

        // Cloning an object that has a nested object with backreference
        // requires special treatment. After the cloning is completed, the
        // nested object should point to the cloned object, instead of the
        // original object.
        $this->circularReference = clone $this->circularReference;
        $this->circularReference->prototype = $this;
    }
}

class ComponentWithBackReference
{
    public $prototype;

    /**
     * Note that the constructor won't be executed during cloning. If you have
     * complex logic inside the constructor, you may need to execute it in the
     * `__clone` method as well.
     */
    public function __construct(Prototype $prototype)
    {
        $this->prototype = $prototype;
    }
}

/**
 * The client code.
 */
function clientCode()
{
    $p1 = new Prototype();
    $p1->primitive = 245;
    $p1->component = new \DateTime();
    $p1->circularReference = new ComponentWithBackReference($p1);
}

```

```

    $p2 = clone $p1;
    if ($p1->primitive === $p2->primitive) {
        echo "Primitive field values have been carried over to a clone. Yay!\n";
    } else {
        echo "Primitive field values have not been copied. Booo!\n";
    }
    if ($p1->component === $p2->component) {
        echo "Simple component has not been cloned. Booo!\n";
    } else {
        echo "Simple component has been cloned. Yay!\n";
    }

    if ($p1->circularReference === $p2->circularReference) {
        echo "Component with back reference has not been cloned. Booo!\n";
    } else {
        echo "Component with back reference has been cloned. Yay!\n";
    }

    if ($p1->circularReference->prototype === $p2->circularReference->prototype) {
        echo "Component with back reference is linked to original object. Booo!\n";
    } else {
        echo "Component with back reference is linked to the clone. Yay!\n";
    }
}

clientCode();

```

Output.txt — Execution result / एक्जीक्यूशन रिज़ल्ट:

```

Primitive field values have been carried over to a clone. Yay!
Simple component has been cloned. Yay!
Component with back reference has been cloned. Yay!
Component with back reference is linked to the clone. Yay!

```

Real World Example

The Prototype pattern provides a convenient way of replicating existing objects instead of trying to reconstruct the objects by copying all of their fields directly. The direct approach not only couples you to the classes of the objects being cloned, but also doesn't allow you to copy the contents of the private fields. The Prototype pattern lets you perform the cloning within the context of the cloned class, where the access to the class' private fields isn't restricted.

प्रोटोटाइप पैटर्न, मौजूदा ऑब्जेक्ट्स को रेप्लिकेट (replicate) करने का एक सुविधाजनक (convenient) तरीका देता है, बजाय इसके कि उनकी सभी फ़िल्ड्स को सीधे कॉपी करके ऑब्जेक्ट्स को दोबारा बनाने की कोशिश की जाए। डायरेक्ट तरीका न सिर्फ़ आपको क्लोन किए जा रहे ऑब्जेक्ट्स की क्लासेज़ से जोड़ (couple) देता है, बल्कि यह प्राइवेट फ़िल्ड्स की सामग्री (contents) कॉपी करने की अनुमति भी नहीं देता। प्रोटोटाइप पैटर्न आपको क्लोन की जा रही क्लास के संदर्भ (context) में ही क्लोनिंग करने देता है, जहाँ क्लास के प्राइवेट फ़िल्ड्स तक पहुँच प्रतिबंधित (restricted) नहीं होती।

This example shows you how to clone a complex Page object using the Prototype pattern. The Page class has lots of private fields, which will be carried over to the cloned object thanks to the Prototype pattern.

यह उदाहरण दिखाता है कि प्रोटोटाइप पैटर्न का उपयोग करके एक कॉम्प्लेक्स Page ऑब्जेक्ट को कैसे क्लोन किया जाए। Page क्लास में ढेर सारे प्राइवेट फ़िल्ड्स हैं, जो प्रोटोटाइप पैटर्न की बदौलत क्लोन किए गए ऑब्जेक्ट में भी मौजूद रहेंगे।

index.php — Real world example (verbatim from source / स्रोत से ज्यों-का-त्यों):

```

<?php

namespace RefactoringGuru\Prototype\RealWorld;

/**
 * Prototype.
 */
class Page
{
    private $title;

    private $body;

    /**
     * @var Author
     */
    private $author;

    private $comments = [];

    /**
     * @var \DateTime
     */
    private $date;

    // +100 private fields.

    public function __construct(string $title, string $body, Author $author)
    {
        $this->title = $title;
        $this->body = $body;
        $this->author = $author;
        $this->author->addToPage($this);
        $this->date = new \DateTime();
    }

    public function addComment(string $comment): void
    {
        $this->comments[] = $comment;
    }

    /**
     * You can control what data you want to carry over to the cloned object.
     *
     * For instance, when a page is cloned:
     * - It gets a new "Copy of ..." title.
     * - The author of the page remains the same. Therefore we leave the
     * reference to the existing object while adding the cloned page to the list
     * of the author's pages.
     * - We don't carry over the comments from the old page.
     * - We also attach a new date object to the page.
     */
    public function __clone()
    {
        $this->title = "Copy of " . $this->title;
        $this->author->addToPage($this);
        $this->comments = [];
        $this->date = new \DateTime();
    }
}

```

```

    }
}

class Author
{
    private $name;

    /**
     * @var Page[]
     */
    private $pages = [];

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    public function addToPage(Page $page): void
    {
        $this->pages[] = $page;
    }
}

/**
 * The client code.
 */
function clientCode()
{
    $author = new Author("John Smith");
    $page = new Page("Tip of the day", "Keep calm and carry on.", $author);

    // ...

    $page->addComment("Nice tip, thanks!");

    // ...

    $draft = clone $page;
    echo "Dump of the clone. Note that the author is now referencing two objects.\n\n";
    print_r($draft);
}

clientCode();

```

Output.txt — Execution result / एक्जीक्यूशन रिज़ल्ट:

Dump of the clone. Note that the author is now referencing two objects.

```
RefactoringGuru\Prototype\RealWorld\Page Object
(
  [title:RefactoringGuru\Prototype\RealWorld\Page:private] => Copy of Tip of the day
  [body:RefactoringGuru\Prototype\RealWorld\Page:private] => Keep calm and carry on.
  [author:RefactoringGuru\Prototype\RealWorld\Page:private] =>
RefactoringGuru\Prototype\RealWorld\Author Object
  (
    [name:RefactoringGuru\Prototype\RealWorld\Author:private] => John Smith
    [pages:RefactoringGuru\Prototype\RealWorld\Author:private] => Array
      (
        [0] => RefactoringGuru\Prototype\RealWorld\Page Object
          (
            [title:RefactoringGuru\Prototype\RealWorld\Page:private] =>
Tip of the day
            [body:RefactoringGuru\Prototype\RealWorld\Page:private] =>
Keep calm and carry on.
            [author:RefactoringGuru\Prototype\RealWorld\Page:private]
=> RefactoringGuru\Prototype\RealWorld\Author Object
            *RECURSION*
            [comments:RefactoringGuru\Prototype\RealWorld\Page:private]
=> Array
              (
                [0] => Nice tip, thanks!
              )
            [date:RefactoringGuru\Prototype\RealWorld\Page:private] =>
DateTime Object
              (
                [date] => 2018-06-04 14:50:39.306237
                [timezone_type] => 3
                [timezone] => UTC
              )
            [1] => RefactoringGuru\Prototype\RealWorld\Page Object
            *RECURSION*
          )
        )
      )
    [comments:RefactoringGuru\Prototype\RealWorld\Page:private] => Array
      (
      )
    [date:RefactoringGuru\Prototype\RealWorld\Page:private] => DateTime Object
      (
        [date] => 2018-06-04 14:50:39.306272
        [timezone_type] => 3
        [timezone] => UTC
      )
  )
)
```

Technical Words Glossary / तकनीकी शब्दों की शब्दावली

Every important technical term used in this document, with its Hindi translation and a short example. इस दस्तावेज़ में इस्तेमाल हुए हर महत्वपूर्ण तकनीकी शब्द का हिंदी अनुवाद और एक छोटा उदाहरण।

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Design Pattern	डिज़ाइन पैटर्न	Prototype एक डिज़ाइन पैटर्न है जो ऑब्जेक्ट कॉपी करने का बार-बार आने वाला समाधान (recurring solution) देता है।
Creational Pattern	क्रिएशनल पैटर्न	Singleton, Factory, Builder और Prototype — ये सभी क्रिएशनल पैटर्न हैं, जो ऑब्जेक्ट बनाने से जुड़े हैं।
Prototype	प्रोटोटाइप	<code>\$original</code> एक प्रोटोटाइप है — इसे <code>clone \$original</code> से कॉपी किया जा सकता है।
Clone (verb/ keyword)	क्लोन करना / clone कीवर्ड	PHP में <code>\$copy = clone \$original;</code> लिखकर एक ऑब्जेक्ट को क्लोन किया जाता है।
Object	ऑब्जेक्ट	<code>new Circle()</code> से बना हर instance एक ऑब्जेक्ट है।
Class	क्लास	<code>Circle</code> और <code>Rectangle</code> , <code>Shape</code> की सबक्लासेज़ (subclasses) हैं।
Interface	इंटरफ़ेस	Prototype इंटरफ़ेस सिर्फ़ <code>clone()</code> मेथड डिक्लेयर करता है, कोई इम्प्लीमेंटेशन नहीं देता।
Concrete Class	कॉन्क्रीट क्लास (असली क्लास)	<code>Rectangle</code> एक कॉन्क्रीट क्लास है — इसे सीधे <code>new Rectangle()</code> से बनाया जा सकता है।
Field / Property	फ़ील्ड / प्रॉपर्टी	<code>\$title</code> , <code>\$body</code> , <code>\$author</code> — ये सभी <code>Page</code> क्लास की फ़ील्ड्स हैं।
Private Field	प्राइवेट फ़ील्ड	<code>private \$title;</code> — यह फ़ील्ड क्लास के बाहर से सीधे एक्सेस नहीं की जा सकती।
Constructor	कंस्ट्रक्टर	<code>__construct()</code> मेथड, ऑब्जेक्ट बनते समय सबसे पहले चलता है।
Subclass	सबक्लास	<code>Circle extends Shape</code> में <code>Circle</code> , <code>Shape</code> की सबक्लास है।
Superclass / Parent Class	सुपरक्लास / पैरेंट क्लास	<code>Shape</code> , <code>Circle</code> और <code>Rectangle</code> दोनों की पैरेंट क्लास है।
Client Code	क्लाइंट कोड	<code>clientCode()</code> फ़ंक्शन वह कोड है जो <code>Prototype</code> क्लास का उपयोग करता है।
Coupling / Dependency	कपलिंग / डिपेंडेंसी	अगर क्लाइंट कोड को हर बार <code>Circle</code> या <code>Rectangle</code> का नाम लिखना पड़े, तो यह उन क्लासेज़ से टाइट कपलिंग (tight coupling) है।
Instantiate	इंस्टैंशिएट करना (ऑब्जेक्ट बनाना)	<code>new Author("John Smith")</code> — यह <code>Author</code> क्लास को इंस्टैंशिएट कर रहा है।

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Method	मैथड	<code>addComment()</code> एक मैथड है जो <code>Page</code> क्लास में डिफ़ाइन है।
Method Overloading	मैथड ओवरलोडिंग	कुछ भाषाएँ (languages) एक ही नाम के कई कंस्ट्रक्टर्स को अलग-अलग पैरामीटर्स के साथ लिखने देती हैं — इसे मैथड ओवरलोडिंग कहते हैं (PHP इसे सीधे सपोर्ट नहीं करता)।
Registry	रजिस्ट्री	एक <code>PrototypeRegistry</code> , <code>"circle" => \$circlePrototype</code> जैसी एंट्रीज़ स्टोर करती है।
Hash Map	हैश मैप	PHP का associative array (<code>["key" => "value"]</code>) एक हैश मैप की तरह काम करता है।
Circular Reference	सर्कुलर रेफरेंस	जब <code>\$a->b = \$b</code> और <code>\$b->a = \$a</code> हो — दोनों ऑब्जेक्ट्स एक-दूसरे को रेफर (refer) करते हैं।
Inheritance	इनहेरिटेन्स	<code>class Rectangle extends Shape</code> — यहाँ <code>Rectangle</code> , <code>Shape</code> से इनहेरिटेन्स के ज़रिए फ़ील्ड्स और मैथड्स पाता है।
Polymorphism	पॉलिमॉर्फिज़्म	जब <code>\$shape->clone()</code> कॉल होता है, तो चाहे <code>\$shape</code> असल में <code>Circle</code> हो या <code>Rectangle</code> , सही <code>clone()</code> मैथड ही चलता है — यही पॉलिमॉर्फिज़्म है।
Factory Method	फ़ैक्टरी मैथड	एक मैथड जो अंदर ही अंदर तय करता है कि किस क्लास का ऑब्जेक्ट बनाना है, जैसे <code>createShape(\$type)</code> ।
Abstract Factory	ऐबस्ट्रैक्ट फ़ैक्टरी	एक इंटरफ़ेस जो कई जुड़ी हुई (related) फ़ैक्टरी मैथड्स को एक साथ ग्रुप करता है।
Builder	बिल्डर	एक पैटर्न जो कॉम्प्लेक्स ऑब्जेक्ट को स्टेप-बाय-स्टेप (step-by-step) बनाता है, जैसे <code>\$builder->setWidth(10)->setHeight(20)->build()</code> ।
Singleton	सिंगलटन	एक क्लास जिसका पूरे एप्लिकेशन में सिर्फ़ एक ही इंस्टेंस मौजूद हो सकता है।
Composite	कम्पोज़िट	एक पैटर्न जो ऑब्जेक्ट्स को ट्री (tree) जैसी संरचना में जोड़ता है, जैसे फ़ोल्डर के अंदर फ़ाइलें और फ़ोल्डर।
Decorator	डेकोरेटर	एक पैटर्न जो किसी ऑब्जेक्ट में नई क्षमताएँ (capabilities) जोड़ता है, बिना उसकी क्लास बदले।
Memento	मेमेंटो	एक पैटर्न जो किसी ऑब्जेक्ट की पुरानी स्टेट को सेव करता है, ताकि बाद में उसे वापस (restore) लाया जा सके।
Command	कमांड	एक पैटर्न जो किसी एक्शन (action) को एक ऑब्जेक्ट के रूप में लपेटता (wrap) है, ताकि उसे बाद में चलाया, अंडू (undo) किया या हिस्ट्री में रखा जा सके।
State	स्टेट	ऑब्जेक्ट के अंदर मौजूद डेटा की मौजूदा स्थिति — जैसे <code>Order</code> की स्टेट "Pending" से "Shipped" में बदल सकती है।
Configuration	कॉन्फ़िगरेशन	ऑब्जेक्ट को इस्तेमाल के लिए तैयार करने वाली सेटिंग्स — जैसे भाषा (language), थीम (theme), टैक्स रेट (tax rate)।
Recursive Dependency	रिकर्सिव डिपेंडेंसी	जब कोई ऑब्जेक्ट सीधे या घुमा-फिराकर (indirectly) खुद पर ही निर्भर हो जाए।

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Reference	रेफ़रेंस	PHP में जब कोई ऑब्जेक्ट वैरिएबल किसी और वैरिएबल को असाइन (assign) किया जाता है, तो दोनों एक ही ऑब्जेक्ट का रेफ़रेंस रखते हैं, जब तक क्लोन न किया जाए।

General Words Glossary / सामान्य शब्दों की शब्दावली

Beyond the technical terms above, this document also uses everyday English words that a Hindi-primary reader may not know. Each one below is defined with its Hindi meaning and a plain, non-technical example sentence.

ऊपर दिए तकनीकी शब्दों के अलावा, इस दस्तावेज़ में कुछ सामान्य (everyday) अंग्रेज़ी शब्द भी इस्तेमाल हुए हैं, जो हिंदी-प्रधान (Hindi-primary) पाठक को अपरिचित लग सकते हैं। नीचे हर शब्द का हिंदी अर्थ और एक सामान्य (non-technical) उदाहरण वाक्य दिया गया है।

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
Catch (a catch / the catch)	पेच, छिपी हुई अड़चन	"The plan sounds perfect, but there's a catch — it only works on weekends." योजना एकदम सही लगती है, लेकिन इसमें एक पेच है — यह सिर्फ़ वीकेंड पर काम करती है।
Scattered	बिखरा हुआ	"Her books were scattered all over the room." उसकी किताबें पूरे कमरे में बिखरी हुई थीं।
Laborious	मेहनती, कष्टसाध्य	"Cleaning the entire warehouse by hand was a laborious task." पूरे गोदाम को हाथ से साफ़ करना एक मेहनती काम था।
Dummy	नकली, डमी	"The store used a dummy figure to display the new jacket." दुकान ने नई जैकेट दिखाने के लिए एक डमी पुतला इस्तेमाल किया।
Robust	मज़बूत, दमदार	"This bag is robust enough to survive years of daily use." यह बैग रोज़ाना इस्तेमाल के कई सालों तक टिकने जितना मज़बूत है।
Convenient	सुविधाजनक	"It's more convenient to pay by card than to carry cash." कैश ले जाने के बजाय कार्ड से भुगतान करना ज़्यादा सुविधाजनक है।
Tricky	मुश्किल, पेचीदा	"Parallel parking on a busy street can be tricky." व्यस्त सड़क पर पैरेलल पार्किंग करना मुश्किल हो सकता है।
Straightforward	सीधा-सादा, आसान	"The instructions were straightforward, so assembly took ten minutes." निर्देश सीधे-सादे थे, इसलिए जोड़ने में सिर्फ़ दस मिनट लगे।
Reduce	घटाना, कम करना	"We should reduce our expenses this month." इस महीने हमें अपने खर्च घटाने चाहिए।
Duplicate	नक़ल, डुप्लीकेट	"Please don't send a duplicate of the same email." कृपया एक ही ईमेल की डुप्लीकेट न भेजें।
Carried over	आगे ले जाया गया, पहुँचाया गया	"Her enthusiasm from the interview was carried over into her first week at work." इंटरव्यू का उत्साह उसके काम के पहले हफ़्ते तक बना रहा।
Matches (verb)	मेल खाना	"This paint color matches the curtains perfectly." यह पेंट का रंग पर्दों से बिल्कुल मेल खाता है।

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
Dozens	दर्जनों	"Dozens of people showed up for the event." इस इवेंट में दर्जनों लोग आए।
Hundreds	सैकड़ों	"The festival attracted hundreds of visitors." इस त्योहार में सैकड़ों दर्शक आए।
Mass production	बड़े पैमाने पर उत्पादन	"The factory switched to mass production once the design was finalized." डिज़ाइन फ़ाइनल होते ही फैक्ट्री बड़े पैमाने पर उत्पादन में बदल गई।
Passive	निष्क्रिय, पैसिव	"He took a passive role in the discussion, mostly just listening." उसने चर्चा में एक पैसिव भूमिका निभाई, ज़्यादातर बस सुनता रहा।
Edge case(s)	असामान्य/किनारे की स्थिति	"The bug only showed up in one edge case — an empty file upload." यह बग सिर्फ़ एक असामान्य स्थिति में दिखा — एक खाली फ़ाइल अपलोड।
Untangling	सुलझाना	"It took an hour to untangle the string of lights." लाइट्स की तार सुलझाने में एक घंटा लग गया।
Worth (adjective)	लायक, योग्य	"This restaurant is well worth the long wait." यह रेस्तराँ लंबे इंतज़ार के लायक है।

This document was generated from refactoring.guru/design-patterns/prototype for personal study purposes, preserving the source's section order and content, with a parallel Hindi translation/explanation added under each section, and PHP code included directly in this file.

यह दस्तावेज़ refactoring.guru/design-patterns/prototype से व्यक्तिगत अध्ययन (personal study) के उद्देश्य से बनाया गया है — स्रोत के सेक्शन क्रम और सामग्री को बनाए रखते हुए, हर सेक्शन के नीचे समानांतर (parallel) हिंदी अनुवाद/व्याख्या जोड़ी गई है, और PHP कोड सीधे इसी फ़ाइल में शामिल किया गया है।